Symbolic Execution

Finding bugs

Cristina David

Lecture 12

COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

Today: find one input that breaks it.

```
if x < 0 then y := -x else y := x;
assert y > 0
```

A change of goal

Lectures 8–11: prove $\{P\}\ c\ \{Q\}$ for *every* input.

Today: find one input that breaks it.

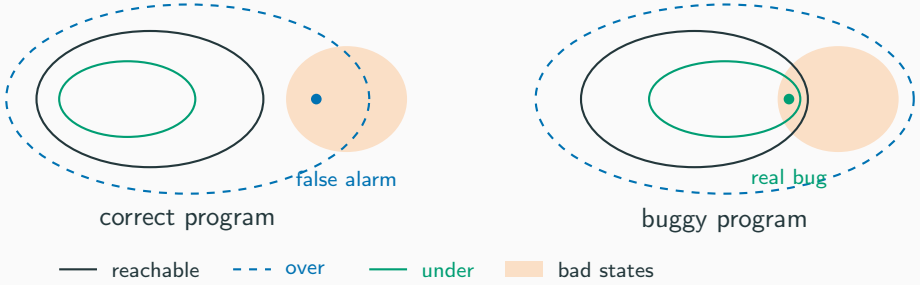
```
if x < 0 then y := -x else y := x;
assert y > 0
```

Is there an input that makes the assertion fail?

Two ways to be wrong

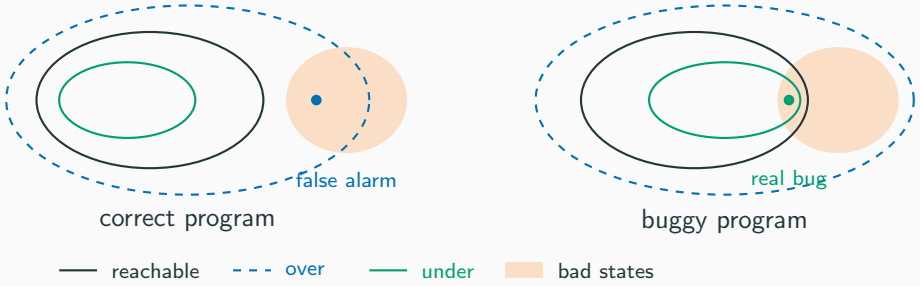
Which way to be wrong?

No analysis computes the reachable states exactly (undecidable). It must over- or under-shoot.



Which way to be wrong?

No analysis computes the reachable states exactly (undecidable). It must over- or under-shoot.



- Over (Hoare logic): silence *proves* no bug; an alarm may be false.
- Under (symbolic execution): an alarm is a *real* bug; silence proves nothing.

What under-approximating forces on you

1. No invariants. An invariant over-approximates. So loops are unrolled, finitely.
2. No merging. Joining branches loses which states are real. So conditionals fork.
3. Witnesses. A state is real only if some input reaches it.

What under-approximating forces on you

1. No invariants. An invariant over-approximates. So loops are unrolled, finitely.
2. No merging. Joining branches loses which states are real. So conditionals fork.
3. Witnesses. A state is real only if some input reaches it.

Together: symbolic execution.

The mechanism

Symbolic inputs

Run the program on a symbolic input $x = \alpha$, not a number. Keep:

- a symbolic store σ: variable $\mapsto$ expression over the inputs;
- a path condition π: what the inputs must satisfy to get *here*.

Symbolic inputs

Run the program on a symbolic input $x = \alpha$, not a number. Keep:

- a symbolic store σ: variable $\mapsto$ expression over the inputs;
- a path condition π: what the inputs must satisfy to get *here*.

Going forwards is the strongest postcondition. For assignment it needs an $\exists$:

$$\mathrm{sp}(y := y + 1,\ y > 0) \ = \ \exists y'.\ y' > 0 \wedge y = y' + 1$$

Symbolic inputs

Run the program on a symbolic input $x = \alpha$, not a number. Keep:

- a symbolic store σ: variable $\mapsto$ expression over the inputs;
- a path condition π: what the inputs must satisfy to get *here*.

Going forwards is the strongest postcondition. For assignment it needs an $\exists$:

$$\mathrm{sp}(y := y + 1,\ y > 0) \ = \ \exists y'.\ y' > 0 \wedge y = y' + 1$$

The symbolic store avoids it: the old value already has a name.

$$\sigma = [\,y \mapsto \beta\,],\ \pi = \beta > 0 \quad \xrightarrow{y:=y+1} \quad \sigma = [\,y \mapsto \beta + 1\,],\ \pi = \beta > 0$$

The execution tree

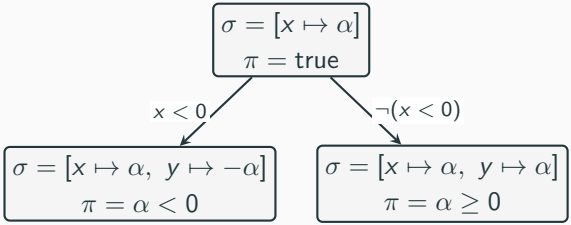
```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```

$$\sigma = [x \mapsto \alpha]$$
$$\pi = \text{true}$$

One leaf per path. Each leaf: a store and the condition to reach it.

The execution tree

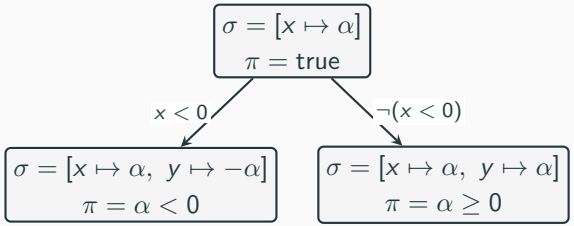
```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



One leaf per path. Each leaf: a store and the condition to reach it.

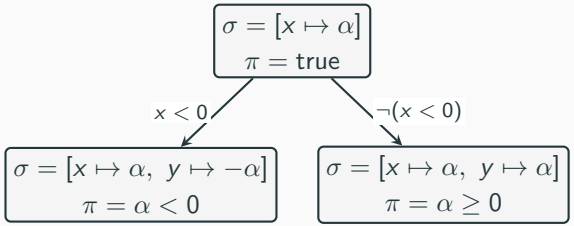
The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



The execution tree

```
if x < 0 then
  y := -x
else
  y := x;
assert y > 0
```



One leaf per path. Each leaf: a store and the condition to reach it.

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds
right	$\alpha \geq 0 \ \wedge \ \alpha \leq 0$	sat, model $\alpha = 0$

Asking the solver

On each leaf: can the assertion fail? Is $\pi \wedge \neg(\sigma(y) > 0)$ satisfiable?

path	query	answer
left	$\alpha < 0 \ \wedge \ -\alpha \leq 0$	unsat: assertion holds
right	$\alpha \geq 0 \ \wedge \ \alpha \leq 0$	sat, model $\alpha = 0$

A test: `x = 0` gives `y = 0`, and the assertion fails.

Fork, don't merge

Hoare's If: one postcondition for both branches

$$\{\text{true}\}\ \texttt{if}\ \ldots\ \{y \geq 0\}$$

Contains $y = 0$, $x = 5$: not reachable.

Symbolic execution: one state per branch

$$\alpha < 0 \wedge y = -\alpha$$
$$\alpha \geq 0 \wedge y = \alpha$$

Every state is reachable.

Fork, don't merge

Hoare's If: one postcondition for both branches

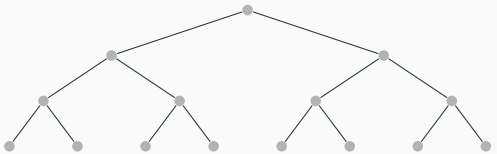
$$\{\text{true}\}\ \texttt{if}\ \ldots\ \{y \geq 0\}$$

Contains $y = 0$, $x = 5$: not reachable.

Symbolic execution: one state per branch

$$\alpha < 0 \wedge y = -\alpha$$
$$\alpha \geq 0 \wedge y = \alpha$$

Every state is reachable.

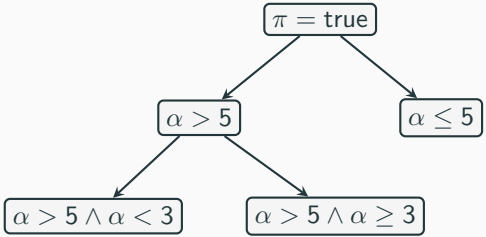


n `ifs` $\Rightarrow$ up to 2^n paths

The price of path-sensitivity: path explosion.

Infeasible paths

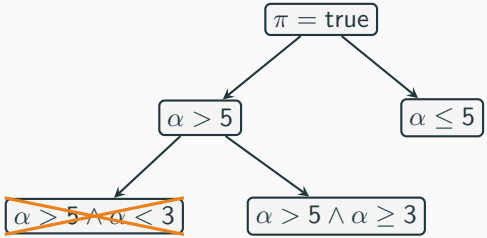
```
if x > 5 then
  if x < 3 then
    error
```



No input reaches `error`. At each fork, ask the solver: is π satisfiable? If not, drop the path.

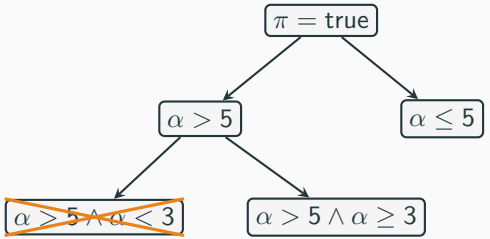
Infeasible paths

```
if x > 5 then
  if x < 3 then
    error
```



Infeasible paths

```
if x > 5 then
  if x < 3 then
    error
```



No input reaches `error`. At each fork, ask the solver: is π satisfiable? If not, drop the path.

Over the heap

The state is a separation-logic formula

$\{\mathsf{list}(x)\}$

```
y := [x.next];
r := [y.val]
```

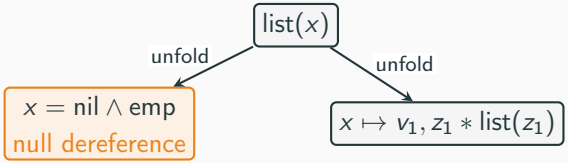
$\boxed{\mathsf{list}(x)}$

Unfolding an inductive predicate is a fork: one branch per case of the definition.

The state is a separation-logic formula

$\{\mathsf{list}(x)\}$

```
y := [x.next];
r := [y.val]
```

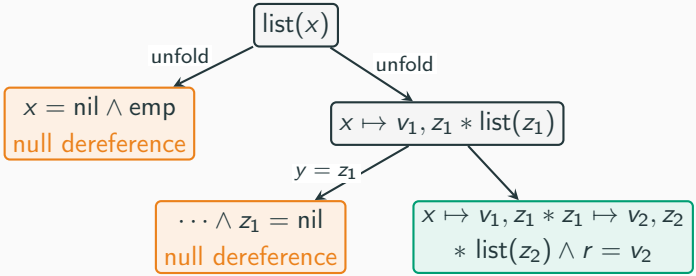


Unfolding an inductive predicate is a fork: one branch per case of the definition.

The state is a separation-logic formula

$\{\mathsf{list}(x)\}$

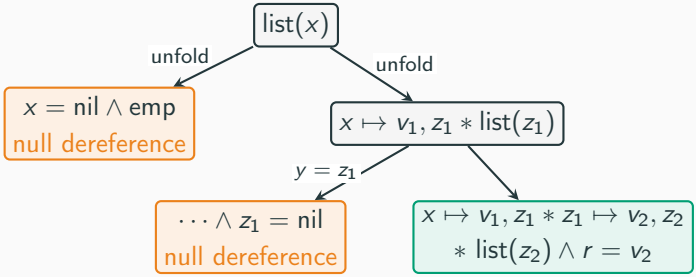
```
y := [x.next];
r := [y.val]
```



The state is a separation-logic formula

$\{\text{list}(x)\}$

```
y := [x.next];
r := [y.val]
```

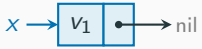


Unfolding an inductive predicate is a fork: one branch per case of the definition.

Each bug comes with a heap



empty list: `[x.next]` faults

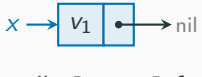


one cell: `[y.val]` faults

Each bug comes with a heap



empty list: `[x.next]` faults



one cell: `[y.val]` faults

Lecture 11's machinery — unfold, step, frame — now driven forwards by an execution engine, not a proof.

Loops: unroll, don't invariant

A loop

```
x := n;
while x > 0 do
  x := x - 2;
assert x = 0
```

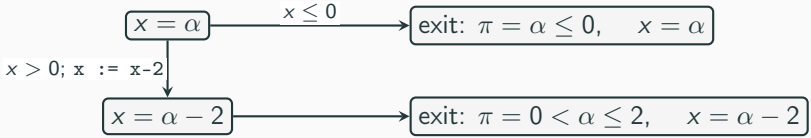
Input $n = \alpha$, symbolic. No invariant: unroll, forking at the guard each lap.

Does the assertion ever fail?

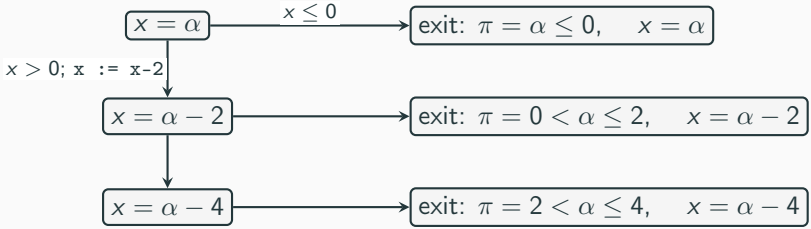
Unrolling, one lap at a time



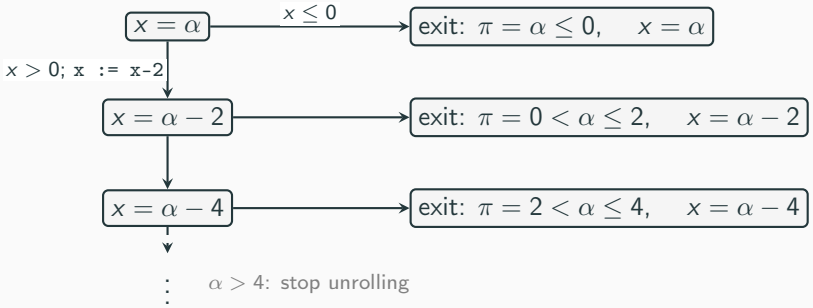
Unrolling, one lap at a time



Unrolling, one lap at a time



Unrolling, one lap at a time



Asking the solver, again

On each exit: is $\pi \wedge \neg(\sigma(x) = 0)$ satisfiable?

query	answer	model
$\alpha \leq 0 \ \wedge \ \alpha \neq 0$	sat	$\alpha = -1$
$0 < \alpha \leq 2 \ \wedge \ \alpha - 2 \neq 0$	sat	$\alpha = 1$
$2 < \alpha \leq 4 \ \wedge \ \alpha - 4 \neq 0$	sat	$\alpha = 3$

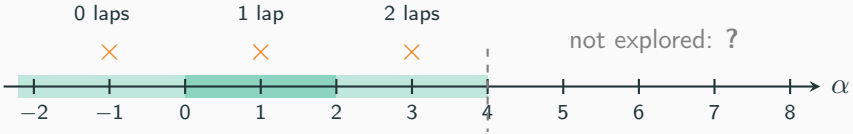
Asking the solver, again

On each exit: is $\pi \wedge \neg(\sigma(x) = 0)$ satisfiable?

query	answer	model
$\alpha \leq 0 \ \wedge \ \alpha \neq 0$	sat	$\alpha = -1$
$0 < \alpha \leq 2 \ \wedge \ \alpha - 2 \neq 0$	sat	$\alpha = 1$
$2 < \alpha \leq 4 \ \wedge \ \alpha - 4 \neq 0$	sat	$\alpha = 3$

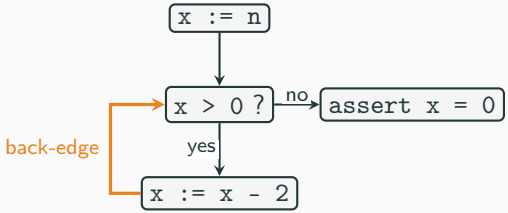
Test `n = 1`: x ends at -1. The loop overshoots on odd input.

What we did *not* learn



- Every bug found is real: each comes with an input.
- About $\alpha > 4$: no claim at all. Silence is not a proof.

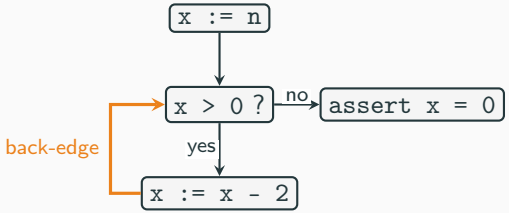
Covering all iterations: abstract at the back-edge



State at the loop head, each time round:

$x = \alpha$ lap 0

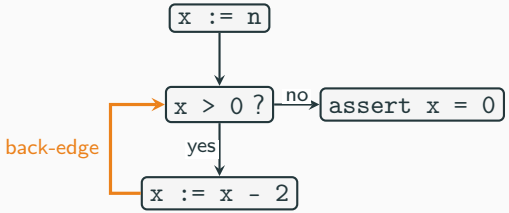
Covering all iterations: abstract at the back-edge



State at the loop head, each time round:

$x = \alpha$	lap 0
$\alpha - 2 \leq x \leq \alpha$	laps ≤ 1
$\alpha - 4 \leq x \leq \alpha$	laps ≤ 2

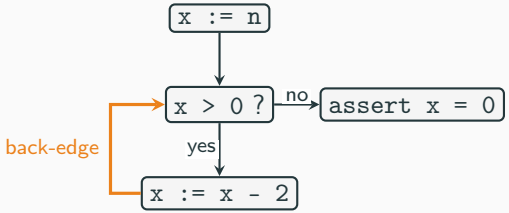
Covering all iterations: abstract at the back-edge



State at the loop head, each time round:

$x = \alpha$	lap 0
$\alpha - 2 \leq x \leq \alpha$	laps ≤ 1
$\alpha - 4 \leq x \leq \alpha$	laps ≤ 2
$x \leq \alpha$	widen: every lap

Covering all iterations: abstract at the back-edge



State at the loop head, each time round:

$x = \alpha$	lap 0
$\alpha - 2 \leq x \leq \alpha$	laps ≤ 1
$\alpha - 4 \leq x \leq \alpha$	laps ≤ 2
$x \leq \alpha$	widen: every lap

At exit: $x \leq 0 \wedge x \leq \alpha$. Cannot show $x = 0$: alarm.

The state also holds unreachable values (e.g. $x = -1$ when $n = 2$): an alarm may be spurious.

Want all laps? Then you over-approximate

Abstracting at the back-edge covers every lap — by over-approximating.

	unroll k times	**abstract at back-edge**
approximates	under	over
reported bugs	always real	may be spurious
silence means	nothing	proof of absence

Sound over-approximation by abstraction: abstract interpretation (Cousot & Cousot 1977).

The whole picture

	Hoare / VCgen	**Symbolic execution**
approximation	over	under
silence means	bugs *proved* absent	nothing found *yet*
an alarm means	maybe a false alarm	a *genuine* bug
loop	invariant	unroll (bounded)
conditional	merge	fork
output	a proof	a test input

Wrap

Today

- Over vs under: prove bugs absent, or report only real bugs.
- Under forces the mechanism: fork, unroll, and a satisfiability query that returns a test input.
- Over the heap, the state is a separation-logic formula; unfolding forks.

Next time

We built a discipline for aliased, mutable memory: $*$ and the frame rule.

What if a language built it into its type system, checked at compile time?

Next time

We built a discipline for aliased, mutable memory: $*$ and the frame rule.

What if a language built it into its type system, checked at compile time?

From Separation Logic to Rust (Lecture 13).

Questions?

ATiPL 2026 Minute Sheet